Introduction to

with Application to Bioinformatics

- Day 2

Yesterday's Key Takeaways

- Liternals, variables
    - Literal: `3.14`, `"fruites"`
    - Variable: `dna`, `myVar`
    - Basic data types: `Numeric`, `String`, `Boolean`, `None` and `Collection`
- Built-in functions and operations
    - `print`, `max`, `sum`
- Loops
    - `for` loop and `while` loop
- Use `if/elif/else` statements to control the logic of the code
- Read and write data from/to file
    - `open(filename, "r")`
    - `open(filename, "w")`

Day 2

- **Session 1**
    - Quiz: Review of Day 1
    - Lecture: Go through the quiz and review the key concepts
- **Session 2**
    - Lecture: Variants identification with VCF
    - Ex1:
    - PyQuiz 2.1
- **Session 3**
    - Lecture: Functions and Methods
    - Ex2:
    - PyQuiz 2.2
    - Ex3: IMDb
- **Project time**

Quiz: Review Day 1

Go to Canvas, `Modules -> Day 2 -> Review Day 1`

~20 minutes

Variables and Types (Q 1 & 2)

1. Which of the following is of the type `float`?:
7.4

2. Match the following variables with their type:

`var1 = 54`	**integer**
`var2 = [1,2,7,1,24]`	**list**
`var3 = 2.98`	**float**
`var4 = True`	**boolean**

Literals

All literals have a type:

- Strings (str) 'Hello' "Hi"
- Integers (int) 5
- Floats (float) 3.14
- Boolean (bool) True or False

In [ ]:
```
type(5)
```

Variables

Used to store values and to assign them a name.

In [ ]:
```
x = "ATGCGTAC"
y = len(x)
print("length of", x, "is", y)
```

Variables

Used to store values and to assign them a name.

In [ ]:
```
x = "ATGCGTAC"
y = len(x)
print("length of", x, "is", y)
```

In [ ]:
```
# Use meaningful variable names !!
dna_sequence = "ATGCGTAC"
sequence_length = len(dna_sequence)
print("length of", dna_sequence, "is", sequence_length)
```

Lists

A collection of values.

In [ ]:

```
x = [1,5,3,7,8]
y = ['a','b','c']
print(type(y))
# print(x + y) # list can have heterogenous data types
```

Comments (Q 3)

3. Which of the following symbols can be used to write comments in your code?

#

Comments (Q 3)

3. Which of the following symbols can be used to write comments in your code?

#

You can also use docstring `"""` to comment out multiple lines of code, although it is not designed for that.

In [ ]:
```
"""This is a description
of the code
below"""
li = ["abc"] * 10
print(li)
```

Operations (Q 4-6)

4. What happens if you do `[1,2,5,11] + [87,2,43,3]`?

`[1,2,5,11,87,2,43,3]` The lists will be concatenated

5. How do you find out if the variable `x` is present in a the list `mylist`?

Two answers correct:

1.

```
x in mylist
```

2.

```
for l in mylist:
        if l == x:
            print('Found a match')
```

6. How do you find out if 5 is larger than 3 and the integer 4 is the same as the float 4? Fill in all the missing code.

```
5 > 3 and 4 == 4.0
```

6. How do you find out if 5 is larger than 3 and the integer 4 is the same as the float 4? Fill in all the missing code.

```
5 > 3 and 4 == 4.0
```

```
5 > 3 and 4 == float(4.0)
```

Basic operations

Type	Operations
int	+ - * / ** % // ...
float	+ - * / ** % // ...
string	+ *

In [ ]:
```
i = 2
f = 5.46
li1 = [1,2,3,4]
li2 = [5,6,7,8]
```

In [ ]:
```
i * f
```

In [ ]:
```
li1 * i
```

Basic operations

Type	Operations
int	+ - * / ** % // ...
float	+ - * / ** % // ...
string	+ *

In [ ]:
```
i = 2
f = 5.46
li1 = [1,2,3,4]
li2 = [5,6,7,8]
```

In [ ]:
```
i * f
```

In [ ]:
```
li1 * i
```

In [ ]:
```
li1 * f
```

Comparison/Logical/Membership operators

Operation	Meaning
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal
==	equal
!=	not equal

Operation	Meaning
and	connects two statements, both conditions having to be fulfilled
or	connects two statements, either conditions having to be fulfilled
not	reverses and/or

Operation	Meaning
in	value in object
not in	value not in object

In [ ]:
```
li = [1,2,3,4,5,6,7,8]
b = 5
b in li
```

In [ ]:
```
b not in li
# not b in li # works as well, but not recommended
```

In [ ]:
```
c = 10
b < c or c == 1
```

Python Operator Precedence (Highest → Lowest)

Priority	Operator Category	Operators	Notes
1	**Parentheses**	`( )`	Force order of evaluation
2	**Exponentiation**	`**`	Right-associative
3	**Unary operators**	`+x`, `-x`, `~x`	Act on one operand
4	**Multiplicative**	`*`, `/`, `//`, `%`	Multiply/divide/ modulo
5	**Additive**	`+`, `-`	Addition and subtraction
6	**Bitwise shifts**	`<<`, `>>`	Shift bits left/right
7	**Bitwise AND**	`&`	
8	**Bitwise XOR**	`^`	
9	**Bitwise OR**	`|`	
10	**Comparisons**	`==`, `!=`, `<`, `<=`, `>`, `>=`, `is`, `is not`, `in`, `not in`	Chainable
11	**Boolean NOT**	`not`	Unary boolean
12	**Boolean AND**	`and`	
13	**Boolean OR**	`or`	

More details: https://www.w3schools.com/python/python_operators_precedence.asp

Sequences and collections (Q 7-9)

7. How do you select the second element in the variable `mylist = [4,3,8,10]`?

Sequences and collections (Q 7-9)

7. How do you select the second element in the variable `mylist = [4,3,8,10]`?

Lists (and strings) are an ORDERED collection of elements where every element can be access through an index.

```
value: 4 3 8 10
index: 0 1 2 3
```

Sequences and collections (Q 7-9)

7. How do you select the second element in the variable `mylist = [4,3,8,10]`?

Lists (and strings) are an ORDERED collection of elements where every element can be access through an index.

```
value: 4 3 8 10
index: 0 1 2 3
```

```
mylist[1]
```

8. Pair the following variables with whether they are mutable or immutable

`var1 = 'my pretty string'`	**immutable**
`var2 = [1,2,3,4,5]`	**mutable**
`var3 = "hello world"`	**immutable**
`var4 = ['a', 'b', 'c', 'd']`	**mutable**

9. Which of the following types are iterable?

Lists and strings

In [ ]:

```
a = [1,2,3,4,5]
b = ['a','b','c']
c = 'a random string'

c[2]
c[1:4]
```

Mutable / Immutable sequences and iterables

Lists are mutable object, meaning you can use an index to change the list, while strings are immutable and therefore not changeable.

An iterable sequence is anything you can loop over, ie, lists and strings.

In [ ]:
```
li1 = ['a','b','c']        # mutable
str1 = 'abc'               # immutable
```

In [ ]:
```
li1[2] = 'B'
li1
```

In [ ]:
```
str1[2] = 'B'
```

New data type: `tuples`

- A tuple is almost like a list but an immutable sequence of objects
- Elements can NOT be changed in a tuple by using its index
- Still iterable

In [ ]:
```
myTuple = (1,2,3,4,'a','b','c')
myList = [1,2,3,4,'a','b','c']
```

In [ ]:
```
for item in myTuple:
    print(item)
```

In [ ]:
```
for item in myList:
    print(item)
```

In [ ]:
```
myList[2] = 42
myList
```

In [ ]:
```
myTuple[2] = 42
```

What is the benefit of `tuple`

- Immutability ensures data integrity and safety, in the use cases when data should be kept unchanged during processing
- Use less memory than `list`, better performance

If/ Else statements (Q 10)

10. How do you do to print 'Yes' if x is bigger than y?

```
if x > y:
    print('Yes')
```

If/ Else statements (Q 10)

10. How do you do to print 'Yes' if x is bigger than y?

```
if x > y:
    print('Yes')
```

In [ ]:

```
a = 3
b = [1,2,3,4]
if a in b:
    print(str(a)+' is found in the list b')
else:
    print(str(a)+' is not in the list')
```

If/ Else statements (Q 10)

10. How do you do to print 'Yes' if x is bigger than y?

```
if x > y:
    print('Yes')
```

In [ ]:
```
a = 3
b = [1,2,3,4]
if a in b:
    print(str(a)+' is found in the list b')
else:
    print(str(a)+' is not in the list')
```

In [ ]:
```
# elif
```

Files and loops (Q 11 & 12)

11. How do you open a file handle to read a file called ‘somerandomfile.txt’?

```
fh = open('somerandomfile.txt')
```

12. The file in the previous question contains several lines, how do you print each line, with or without trailing whitespace?

1. ```
   for line in fh:
       print(line)
   ```

2. ```
   for row in fh:
       print(row.strip())
   ```

In [ ]:

```
fh = open('../files/somerandomfile.txt','r', encoding = 'utf-8')
for line in fh:
    print(line.strip())
fh.close()
```

In [ ]:

```python
# while loop
key = ""
while key != "python2025":
    key = input("Enter your key: ")

print("Correct key!")
```

Questions?

Break (10 min)

Session 2

- Lecture: **Variants identification with VCF**
- Ex1:
- PyQuiz 2.1

Lunch

Variants identification with VCF

- A VCF (Variant Call Format) file is a text file format used in bioinformatics for storing gene sequence variations.

Description of the task

You have a VCF file (`genotypes_small.vcf`, which can be found in the `downloads` folder) with a number of samples. You are interested in only one of the samples (**sample1**) and one region (**chr5, 1,200,000-1,300,000**, including the start and end positions). What you want to know is whether this sample has any variants in this region, and if so, which variants.

What is your input?

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

1. **CHROM**: 1 (Chromosome 1)
2. **POS**: 10177 (Position 10177 on the chromosome, **starting from 1**)
3. **ID**: rs367896725 (Reference SNP ID)
4. **REF**: A (Reference allele is A)
5. **ALT**: AC (Alternative allele is AC)
6. **QUAL**: 50 (Quality score)
7. **FILTER**: PASS (status, meaning passed all quality control)
8. **FORMAT**: GT:DP:CB (Genotype, Depth, Cell Barcode)
9. **SAMPLES**: 0/1:30:SM (Sample genotypes and additional info)

Genotype field

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

`0/1`

- For human, and many other organisms, there are two copies of genetic information, inherited from both parents.
- `0` means DNA at this position is the same as reference allele (`A`),
- `1` means has alternative allele, in this case `AC`

Genotype field

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

`0/1`

- For human, and many other organisms, there are two copies of genetic information, inherited from both parents.
- `0` means DNA at this position is the same as reference allele (`A`),
- `1` means has alternative allele, in this case `AC`

The notation of alternative allele is not always `1` : `0/2`

- `2` means it is the second alternative allele, i.e. `C` in this case

Some terminologies

Genotype	Status	Meaning
0/0	Homozygous Reference (wild type)	Two copies of the reference allele.
0/1 or 1/0	Heterozygous	One copy of the reference allele and one copy of the alternate allele.
1/1	Homozygous Alternate	Two copies of the alternate allele.

Start with pseudocode!

- Pseudocode is a description of what you want to do without actually using proper syntax

Task:

You have a VCF file with a number of samples. You are interested in only one of the samples (sample1) and one region (chr5, 1,200,000-1,300,000, including the start and end positions). What you want to know is whether this sample has any variants in this region, and if so, which variants.

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

- Open the file and loop over lines (ignore lines with started with `#`)
- Identify lines where chromosome is equal to `5` and position is between 1,200,000 and 1,300,000
- Isolate the column that contains the genotype for sample1, e.g. `0/1:30:SM`
- Extract only the genotypes (e.g. `0/1`) from the column
- Check if the genotype contains any alternate alleles, i.e., if not `0/0`.
- Print any variants containing alternate alleles for this sample between specified region

Open the file and loop over lines (ignore lines starting with `#`), and print the first record

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

In [ ]:
```
with open('../downloads/genotypes_small.vcf', 'r', encoding='utf-8') as
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            print(line)
            break
# Next, find chromosome 5
```

In [ ]:
```

```

Identify lines where chromosome is `5`

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r', encoding='utf-8') as
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            if chrom == '5':
                print(cols)
                break

# Next, find the correct region
```

Identify lines where chromosome is `5` and position is between `1,200,000` and `1,300,000`

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r', encoding='utf-8') as
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                print(cols)
                break
# Next, find the genotypes for sample1
```

Isolate the column that contains the genotype for sample1

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r', encoding='utf-8') as
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                gt_field = cols[9]
                print(gt_field)
                break

# Next, extract the genotypes only
```

Extract only the genotypes from the column

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r', encoding='utf-8') as
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                print(gt)
                break

# Next, find in which positions sample1 has alternate alleles
```

Check if the genotype contains any alternate alleles

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r', encoding='utf-8') as
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                if gt != "0/0": # For robust VCF processing, this condit
                    print(gt)
#Next, print nicely
```

Print any variants containing alternate alleles for this sample between specified region

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:SMB  0/1:32:MB
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1:55:SMB  2/3:58:MB
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r', encoding='utf-8') as
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                if gt != "0/0": # For robust VCF processing, this condit
                    ref = cols[3]
                    alt = cols[4]
                    info = chrom + ":" + str(pos) + "_" + ref + "-" + al
                    print(info)
```

Exercise 1

Go to Canvas -> Moudles -> Exercises Day 2

Three options

1. Green exercise
2. Yellow exercise
3. Red exercise

The level of complexity increases with each exercise

- ChatGPT exercise

New to programming: Do Green exercise and possibly Yellow exercise + ChatGPT
More experienced: Do Yellow exercise and/or Red exercise + ChatGPT

The exercise will be continued in the afternoon

Session 3

- Lecture: **Functions and Methods**
- Ex2:
- PyQuiz 2.2
- IMDB exercise

What is a function

- A function is a block of code that performs a specific task.
- It can take input (parameters) and return output (results).
- Functions help to reuse code and make programs more modular and readable.
- With a function, you only need to know how to use it, but not how it's implemented.

In [ ]:
```
print("this is your sequence")
```

In [ ]:
```
sorted([5,3,135,31])
```

What is a method

- A method is a function that is associated with objects (instances of class)
  - All data types in Python are objects

In [ ]:
```
"   I'm a line with trailing spaces  ".strip()
```

In [ ]:
```
"   I'm a line with trailing spaces  ".split()
```

In [ ]:
```
li = [3, 1, 5, 43, 5]
li.sort()
print(li)
```

What does it matter to me?

For now, you only need to know the different syntaxes of using a function and a method and how to use them

A function:
`functionName(arg1, arg2)`

A method:
`obj.methodName()`

Python built-in functions:

https://docs.python.org/3/library/functions.html

Built-in Functions				
abs( )	delattr( )	hash( )	memoryview( )	set( )
all( )	dict( )	help( )	min( )	setattr( )
any( )	dir( )	hex( )	next( )	slice( )
ascii( )	divmod( )	id( )	object( )	sorted( )
bin( )	enumerate( )	input( )	oct( )	staticmethod( )
bool( )	eval( )	int( )	open( )	str( )
breakpoint( )	exec( )	isinstance( )	ord( )	sum( )
bytearray( )	filter( )	issubclass( )	pow( )	super( )
bytes( )	float( )	iter( )	print( )	tuple( )
callable( )	format( )	len( )	property( )	type( )
chr( )	frozenset( )	list( )	range( )	vars( )
classmethod( )	getattr( )	locals( )	repr( )	zip( )
compile( )	globals( )	map( )	reversed( )	__import__( )
complex( )	hasattr( )	max( )	round( )	

Introduction to some useful functions

help

In [ ]:
```
help(max)
```

In [ ]:
```
a = 5
help([1,2,3])
```

range

In [ ]:
```
for i in range(5):
    print(i)
```

range

In [ ]:
```
for i in range(5):
    print(i)
```

In [ ]:
```
# range(start, stop, step)
# generate a series of odd number
for i in range(1, 10, 2):
    print(i)
```

range

In [ ]:
```
for i in range(5):
    print(i)
```

In [ ]:
```
# range(start, stop, step)
# generate a series of odd number
for i in range(1, 10, 2):
    print(i)
```

In [ ]:
```
# negative step
for i in range(10, 1, -2):
    print(i)
```

sorted

In [ ]:
```
li = [5, 4, 1, 3, 10, 13, 13]
print(sorted(li))
```

sorted

In [ ]:
```
li = [5, 4, 1, 3, 10, 13, 13]
print(sorted(li))
```

In [ ]:
```
print(sorted(li, reverse=True))
```

sorted

In [ ]:
```
li = [5, 4, 1, 3, 10, 13, 13]
print(sorted(li))
```

In [ ]:
```
print(sorted(li, reverse=True))
```

In [ ]:
```
# sorted does not change the content in place
li = [5, 4, 1, 3, 10, 13, 13]
print(sorted(li))
print(li)
sorted_li = sorted(li)
print(li)
```

`dir`:

return a list of names comprising the attributes of the given object

In [ ]:
```
dir(str)
```

In [ ]:
```
a = "This is an ongoing course".strip().split().index('an')
dir(a)
```

Some userful methods for Strings

String Methods	
str.strip( )	str.startswith( )
str.rstrip( )	str.endswith( )
str.lstrip( )	str.upper( )
str.split( )	str.lower( )
str.join( )	

str.split()

In [ ]:
```
a = '  split a "string into a list" '
a.split()
```

str.split()

In [ ]:
```
a = '  split a "string into a list" '
a.split()
```

In [ ]:
```
a.split(maxsplit=2)
```

str.lstrip()

`str.`**lstrip**([*chars*])

Return a copy of the string with leading characters removed. The *chars* argument is a string specifying the set of characters to be removed. If omitted or `None`, the *chars* argument defaults to removing whitespace. The *chars* argument is not a prefix; rather, all combinations of its values are stripped:

```
>>> '   spacious   '.lstrip()
'spacious   '
>>> 'www.example.com'.lstrip('cmowz.')
'example.com'
```

In [ ]:
```
line = "TAG:here comes the actual content"
line.lstrip("TAG:")
print(line.lstrip("TAG"))
print(line) # the content of line is not changed in place
```

In [ ]:
```
line = "AB:A:B should not be removed"
line.lstrip("AB:") # The chars argument is not a prefix
```

str.join

In [ ]:
```
','.join(["ATGC", "CCCCC", "AAATG", "CTTTGTA"]) # convert a list to stri
```

Some userful methods for Lists

Operation	Result
s.append(x)	appends *x* to the end of the sequence
s.insert(i, x)	*x* is inserted at pos *i*
s.pop([i])	retrieves the item *i* from *s* and also removes it
s.remove(x)	retrieves the first item from *s* where s[i] == x
s.reverse( )	reverses the items of *s* in place

In [ ]:
```
a = [1, 2, 3, 4, 5, 5, 5, 5]
a.append(6)
a
```

In [ ]:
```
a.pop()
a
```

In [ ]:
```
a.pop(1) # pop(index)
a
```

In [ ]:
```
a.reverse()
a
```

In [ ]:
```
a.remove(5) # remove(value)
a
```

In [ ]:
```
a.insert(0, 13)
a
```

In [ ]:
```
### Tuples are immutable
a = (2, 3, 4, 15)
a.pop()
```

Summary

- Tuples are immutable sequences of objects
- Plan your approach before you start coding
- A method always belongs to an object of a specific class, a function does not have to
    - call a function by `functionName(args)`
    - call a method by `obj.methodName(args)`
- The official Python documentation describes the syntax for all built-in functions and methods
    - https://docs.python.org/3/
    - https://www.w3schools.com/python/

Exercise 2 (continuation of Ex1)

Three options

1. Green exercise
2. Yellow exercise
3. Red exercise

The level of complexity increases with each exercise.

- ChatGPT exercise

New to programming: Do Green exercise and possibly Yellow exercise + ChatGPT
More experienced: Do Yellow exercise and/or Red exercise + ChatGPT

Break (10 min)

PyQuiz 2.2

Go to Canvas, `Modules -> Day 2 -> PyQuiz 2.2`

Exercise with IMDb

Go to Canvas, `Modules -> Day 2 -> IMDb exercise - day 2`

IMDb

Get the file `250.imdb` from the download folder (https://raw.githubusercontent.com/NBISweden/workshop-python/ht25/downloads.zip)

The format of this file is:

- Line by line
- Columns separated by the | character
- Header starting with #

```
# Votes | Rating | Year | Runtime | URL | Genres | Title
  126807|    8.5|1957|5280|https://images-na.ssl-images....|Drama,War|Paths of Glory
   71379|    8.2|1925|4320|https://images-na.ssl-images....|Adventure,Comedy,Drama,Family|The Gold
```

Votes | Rating | Year | Runtime | URL | Genres | Title

Task: Find the movie with the highest rating

```
# Votes | Rating | Year | Runtime | URL | Genres | Title
   126807|    8.5|1957|5280|https://images-na.ssl-images....|Drama,War|Paths of Glory
    71379|    8.2|1925|4320|https://images-na.ssl-images....|Adventure,Comedy,Drama,Family|The Gold
```

Write step-by-step pseudocode

```
# Votes | Rating | Year | Runtime | URL | Genres | Title
   126807|    8.5|1957|5280|https://images-na.ssl-images....|Drama,War|Paths of Glory
    71379|    8.2|1925|4320|https://images-na.ssl-images....|Adventure,Comedy,Drama,Family|The Gold
```

- Open file
- Initiate counter to keep track of highest rating and movie. Start counter at 0
- Loop over all lines not starting with '#'
- Strip and split the lines into a list
- Save the element containing the rating from the list into a variable
- If current rating is higher than the rating in the counter, replace counter value with rating and movie
- Close file
- Print counter

```
# Votes | Rating | Year | Runtime | URL | Genres | Title
   126807|   8.5|1957|5280|https://images-na.ssl-images....|Drama,War|Paths of Glory
    71379|   8.2|1925|4320|https://images-na.ssl-images....|Adventure,Comedy,Drama,Family|The Gold
```

In [ ]:

```
fh   = open('../downloads/250.imdb', 'r', encoding='utf-8')
best = [0,'']           # here we save the rating and which movie
for line in fh:
    if not line.startswith('#'):
        cols   = line.strip().split('|')
        rating = float(cols[1].strip())
        if rating > best[0]:           # if the rating is higher than pr
            best = [rating,cols[6]]
fh.close()
print(best)
```